

TRABAJO INTEGRADOR MATEMÁTICA

Docentes:

Federico Rodriguez
Eloy Molina

Integrantes:

Ankovic, Angelina
Carranza, Carolina
Negro, Delfina
Serafini, Renata

Consigna 1 — Validación de usuarios y análisis de consistencia del sistema

PARTE A — Análisis con conjuntos

1. Calcular e interpretar:

- a) Usuarios que utilizan ambas plataformas:

$$A \cap B = \{104, 105, 106\}$$

- b) Usuarios que utilizan al menos una plataforma:

$$A \cup B = \{101, 102, 103, 104, 105, 106, 107, 108\}$$

- c) Usuarios que utilizan la plataforma, pero no presentan errores:

$$(A \cup B) - C = \{101, 103, 104, 106, 107, 108\}$$

- d) Usuarios que utilizan exclusivamente una sola plataforma:

$$A \Delta B = \{101, 102, 103, 107, 108\}$$

2. Expresar al menos dos resultados usando comprensión de conjuntos:

$$\{x \in \mathbb{N} \mid x \in A \wedge x \in B\}$$

$$\{x \in \mathbb{N} \mid x \in A \vee x \in B\}$$

3. Detectar:

- a) Usuarios que aparecen en C pero no en $A \cup B$:

$$C - (A \cup B) = \{109\}$$

Parte B — Lógica proposicional

5. Construir la tabla de verdad:

p	q	r	$p \vee q$	$(p \vee q) \wedge r$
V	V	V	V	V
V	V	F	V	F
V	F	V	V	V

V	F	F	V	F
F	V	V	V	V
F	V	F	V	F
F	F	V	F	F
F	F	F	F	F

6. Implementar una función en Python que evalúe la expresión:

Primero se definieron tres listas: **A**, **B** y **C**. La lista **A** representa los usuarios que acceden por API, la lista **B** los usuarios que acceden por web y la lista **C** los usuarios que generaron errores en el sistema. Después, se creó una lista llamada **usuarios** que contiene todos los IDs registrados. Luego, mediante un ciclo **for**, el programa recorre cada usuario de la lista para analizar su comportamiento dentro del sistema.

Para cada usuario, se crean tres variables booleanas:

- **p**: verifica si el usuario pertenece a A
- **q**: verifica si pertenece a B
- **r**: verifica si pertenece a C

Luego se evalúa la condición lógica: $(p \vee q) \wedge r$

Esto significa que un usuario será considerado crítico si utiliza la plataforma (API o web) y además generó errores en el sistema.

Finalmente, usando una estructura **if**, el programa muestra en pantalla si cada usuario es “Crítico” o “No crítico” según el resultado de la condición lógica.

LINK DEL REPOSITORIO:

https://github.com/caarocarranza/TP_INTEGRADOR_I/blob/delfina/usuarios_criticos.PY

7. Clasificar los usuarios del dataset en:

- Críticos
- No críticos

Usuarios	$(p \vee q) \wedge r$	Clasificación de Usuarios
101	F	No Crítico
102	V	Crítico
103	F	No Crítico
104	F	No Crítico
105	V	Crítico
106	F	No Crítico
107	F	No Crítico
108	F	No Crítico
109	F	No Crítico

Parte C — Análisis

8. Responder:

- ¿Qué tipo de usuario representa mayor riesgo?

Los usuarios críticos son los que representan un mayor riesgo ya que pertenecen a $C(r)$, es decir, los usuarios que han generado errores (ID 's: 102-105). Esto puede afectar el funcionamiento del sistema, provocar fallos y generar problemas para otros usuarios.

- ¿Qué significa que un usuario esté en C pero no en $A \cup B$?

Significa que el usuario presenta errores registrados, pero no aparece utilizando ninguna de las plataformas. En este caso, es el ID: 109.

- ¿Qué decisión tomarían como equipo programador?

Como equipo programador, nosotros prestaríamos más atención a los usuarios críticos, porque son los que cumplen la condición de riesgo.

Podríamos controlar mejor sus acciones, revisar su actividad y agregar medidas de seguridad para evitar problemas en el sistema.

CONSIGNA 2 - Análisis y optimización de costos de desarrollo

PARTE A - Análisis matemático

1. Determinar pendiente y ordenada al origen de funciones $A(x) = 40x + 200$ y $B(x) = 70x + 50$:

$A(x)$: Pendiente = 40

Ordenada al origen = 200

$B(x)$: Pendiente = 70

Ordenada al origen = 50

2. Indicar si son paralelas

Las funciones A y B NO son paralelas ya que tienen pendientes diferentes.

3. Calcular el punto de intersección entre $A(x) = 40x + 200$ y $B(x) = 70x + 50$:

Coordenada X:

$$40x + 200 = 70x + 50$$

$$40x - 70x = 50 - 200$$

$$-30x = -150$$

$$x = -150 / -30$$

$$x = 5$$

Coordenada Y:

$$Y = 40x + 200$$

$$y = 40 * 5 + 200$$

$$y = 200 + 200$$

$$y = 400$$

El punto de intersección de las funciones A y B se encuentra en las coordenadas $x=5$, $y=400$

4. Función $C(x) = -2x^2 + 80x + 100$

$$a = -2$$

$$b = 80$$

$$c = 100$$

Calcular vértice:

$$X_v = \frac{-b}{2a}$$

$$X_v = \frac{-80}{2 * (-2)}$$

$$X_v = \frac{-80}{-4} \gg X_v = 20$$

$$Y_v = -2 * (20)^2 + 80 * (20) + 100$$

$$Y_v = -2 * (400) + 80 * (20) + 100$$

$$Y_v = -800 + 1600 + 100 \gg Y_v = 900$$

$$X_v = 20, Y_v = 900$$

Calcular raíces:

$$x = -b \pm \frac{\sqrt{b^2 - 4ac}}{2a}$$

$$x = -80 \pm \frac{\sqrt{80^2 - 4 * (-2) * 100}}{2 * (-2)}$$

$$x = -80 \pm \frac{\sqrt{6400 - (-800)}}{-4}$$

$$x_1 = -80 + \frac{\sqrt{7200}}{-4} \gg x_1 = -1.21$$

$$x_2 = -80 - \frac{\sqrt{7200}}{-4} \gg x_2 = 41.21$$

PARTE B - Implementación

5. Programar las funciones en python

Se realizó un programa que calcula y muestra datos sobre funciones lineales o cuadráticas.

Inicia con un menú que permite al usuario elegir entre los dos tipos de funciones para luego permitir ingresar los valores correspondientes a cada función.

Los datos evaluados de cada función son:

FUNCIÓN LINEAL:

- **Pendiente:** Es directamente el valor **m** ingresado por el usuario.
- **Ordenada al origen:** Es directamente el valor **b** ingresado por el usuario.
- **Raíz:** Se usó la fórmula de raíz para función lineal: $-b / m$

- **Si $m \neq 0$:** Se usa la fórmula sin problemas.
- **Si $m = 0$ y $b \neq 0$:** La función es una recta horizontal que flota arriba o abajo del eje X, por lo tanto, el programa le avisa al usuario que **no tiene raíces** (nunca toca el eje).
- **Si $m = 0$ y $b = 0$:** La recta está apoyada justo sobre el eje X, por lo que el programa informa que existen **infinitas raíces**.
- **Comportamiento:** Se analiza si la función es creciente, decreciente o constante teniendo en cuenta el valor de la pendiente:
 - Si m es positiva la función es creciente.
 - Si m es negativa la función es decreciente
 - Si m es 0 la función es constante.
- **Ángulo:** Para precisión y organización se utilizó la función de python “math” que permite conocer el arcotangente de la pendiente m directamente y pasar el valor que originalmente da en radianes, a grados.
- **Dominio:** En todas las funciones lineales el dominio es el conjunto de todos los números reales (-infinito, +infinito).
- **Rango:** Se analiza según el comportamiento de la pendiente:
 - Si la función es **creciente o decreciente ($m \neq 0$)**: el rango abarca todos los números reales (-infinito, +infinito) porque la recta se proyecta infinitamente hacia arriba y hacia abajo.
 - Si la función es **constante ($m = 0$)**, la recta es completamente horizontal y el rango se reduce únicamente al valor de la ordenada al origen (b), ya que la función nunca cambia de altura.

FUNCIÓN CUADRÁTICA:

- **Ordenada al origen:** Es directamente el valor c ingresado por el usuario.
- **Vértice:** Se utiliza la fórmula de vértice en x : $-b / (2 * a)$
- **Concavidad:** Se analiza el signo del coeficiente a :
 - Si $a > 0$, la parábola es cóncava hacia arriba.
 - Si $a < 0$, la parábola es cóncava hacia abajo.
- **Raíces:** Se calculan evaluando el discriminante.
 - Si es mayor a 0: El programa calcula las **dos raíces reales** distintas, las calcula y las muestra.
 - Si es igual a 0: Informa que tiene **una sola raíz**, el vértice que toca el eje X, la calcula y la muestra.
 - Si es menor a 0: Informa que **no tiene raíces reales** (gráficamente no toca el eje X, da raíces imaginarias).

- **Dominio:** Al igual que la lineal, matemáticamente no tiene restricciones a menos que se aplique en un problema real, que no es el caso de este programa, por lo que abarca todos los números reales (-infinito, +infinito)
- **Rango:** Depende de la concavidad y de la altura del vértice (v_y).
 - Si es cóncava hacia arriba: El rango va desde el vértice al infinito positivo (v_y , +infinito),
 - Si es cóncava hacia abajo: El rango viene del infinito negativo hasta la altura del vértice (-infinito, v_y).

SEGUNDO MENÚ INTERACTIVO:

Se inicia con un bucle `while` $\neq 3$ para que el menú siga apareciendo mientras el usuario siga eligiendo otras opciones

1. **Generar gráfico de la función:** Para ello se implementaron librerías de python
2. **Obtener valor de Y ingresando valor de X:** Se pide valor de x y se reemplaza x de la ecuación generada por el usuario por el nuevo valor.
3. **Salir del programa:** Sale del bucle `while` ya que genera la condición `while == 3`.

RECURSOS UTILIZADOS:

El programa fue desarrollado usando bloques `if - elif - else` para evaluar situaciones y manejar la opción elegida por el usuario. Se usaron bloques `while` para que el programa se siga ejecutando hasta que el usuario desee salir.

En esta actividad se utilizó la Inteligencia Artificial Gemini para pulir la lógica que el programa debía seguir, para ayudar a corregir errores de ciertos parámetros ingresados por el usuario y para la comprensión e implementación de bibliotecas útiles de python.

Bibliotecas utilizadas:

math:

Es una librería estándar de python que tiene funciones matemáticas avanzadas ya listas para usar. En este caso, se implementó para calcular el ángulo de la función lineal. Su ventaja es la precisión y el ahorro de código y tiempo de crear cálculos desde cero cuando ya existen dentro de python. Solo sirve para cálculos individuales y no para procesar listas grandes de números a la vez.

numpy:

Esta librería es muy útil en el manejo de datos numéricos masivos. Se utilizó para generar de forma matemática el rango y una lista de 100 números continuos para el eje X. Su ventaja es la velocidad y el uso en conjunto con la librería matplotlib para generar gráficos.

matplotlib.pyplot:

El módulo .pyplot indica que se usa esa subcarpeta específica para dibujos simples de dos dimensiones dentro de la gran librería matplotlib.

Es una biblioteca muy usada para graficar funciones matemáticas. Agarra los puntos generados por la biblioteca numpy, los dibuja en la pantalla y los une con una línea para formar el gráfico final.

Abre una ventana que permite moverse por los ejes, hacer zoom o guardar el gráfico como una foto.

Se eligió esta porque era de las más sencillas de implementar en programas sencillos como el que requería.

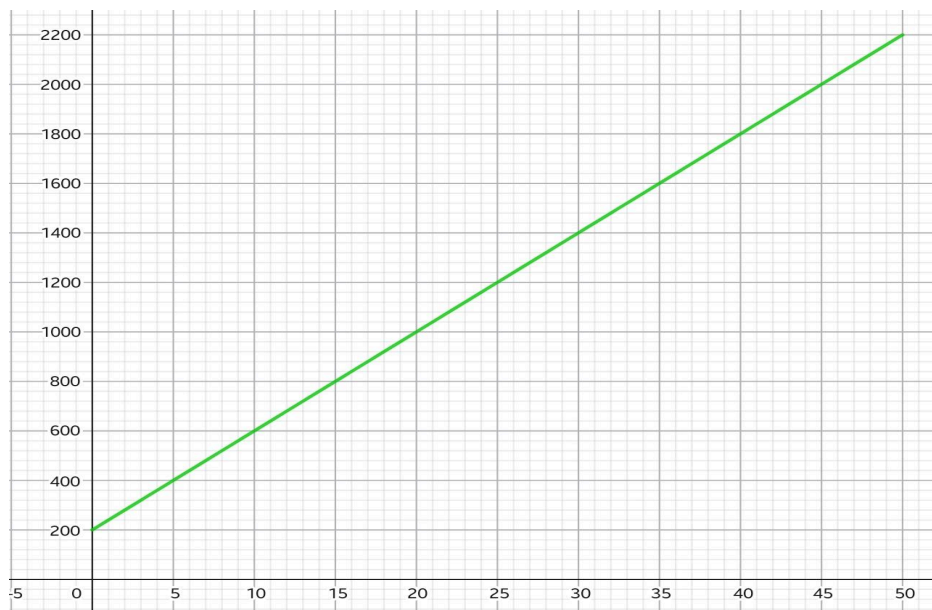
Lógica de bloque de generación de gráficos:

Línea	Explicación
<code>x = np.linspace(-10, 10, 100)</code>	Se genera una lista con 100 números perfectamente distribuidos entre el -10 y el 10 para actuar como los valores del eje X.
Para recta: <code>y = m * x + bLineal</code> Para parábola: <code>y = a * (x**2) + bCuadratica * x + c</code>	Utilizando los coeficientes previamente ingresados por el usuario, Python calcula de manera simultánea el valor de Y para cada uno de esos 100 puntos utilizando las ecuaciones correspondientes a la función.
Para recta: <code>plt.plot(x, y, label="Función Lineal", color="red")</code> Para parábola: <code>plt.plot(x, y, label="Función Cuadrática", color="blue")</code>	El comando <code>plt.plot(x, y)</code> funciona como un "conector de puntos": ubica las 100 coordenadas (x, y) invisibles en la pantalla y las une mediante segmentos rectos muy pequeños. Debido a la gran cantidad de puntos juntos, se percibe una curva o recta perfecta. El argumento color define el color del trazado de la línea. El argumento label le asigna una etiqueta de texto a cada gráfico.

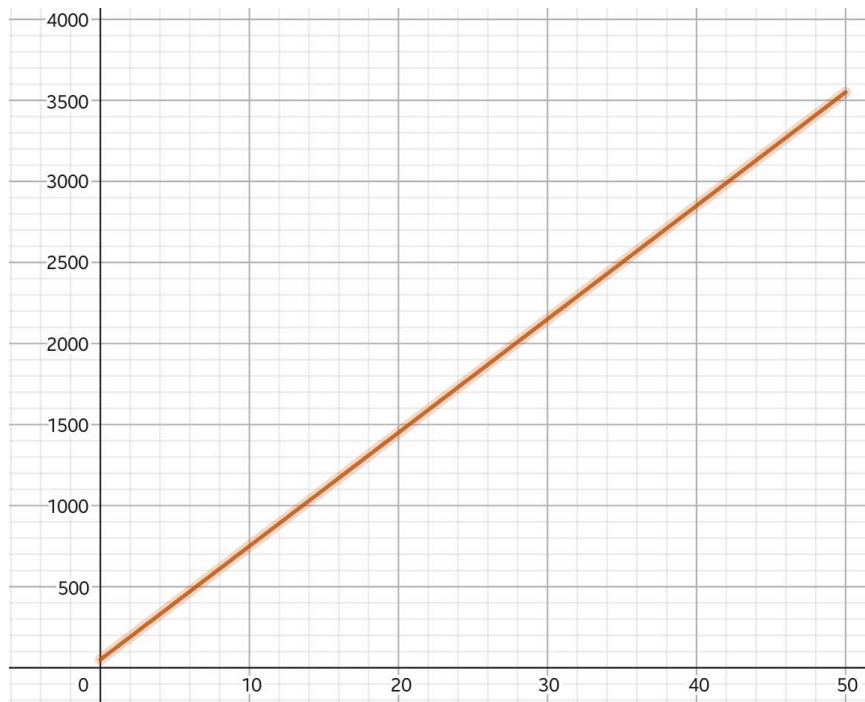
<code>plt.plot(vx, vy, 'ro', label=f"Vértice ({vx:.2f}, {vy:.2f}))"</code>	En el caso de la función cuadrática, se utilizó además la instrucción <code>plt.plot(vx, vy, 'ro')</code> para marcar específicamente el vértice como un punto rojo ('ro') destacado.
<code>plt.show()</code>	Renderiza y abre la ventana en el sistema operativo, pausando la terminal de comandos hasta que el usuario decida cerrar la imagen.

6. Graficar en el intervalo dado:

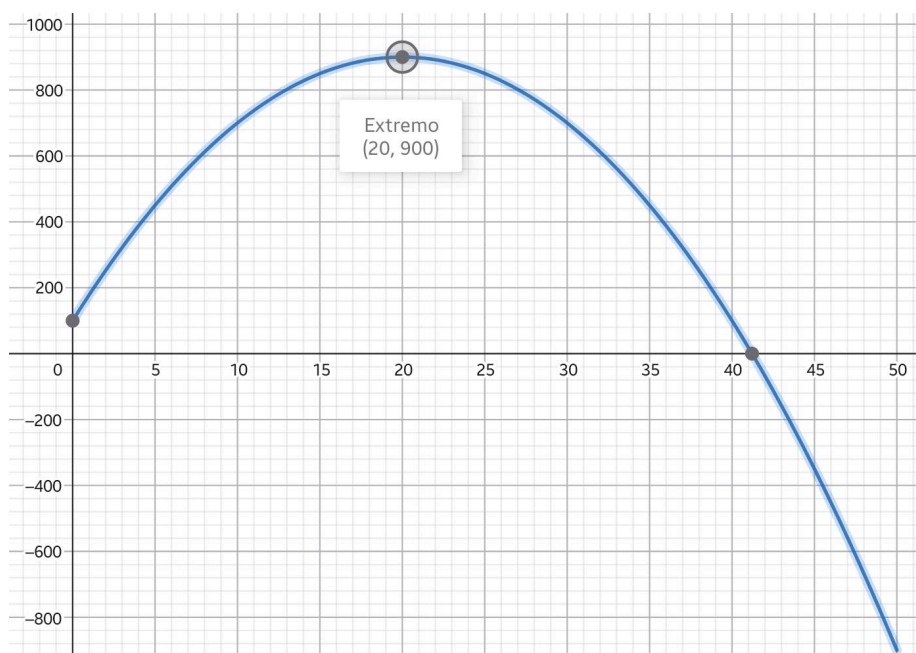
$A(x) = 40x + 200$ en un intervalo de $0 \leq x \leq 50$



$B(x) = 70x + 50$ en un intervalo de $0 \leq x \leq 50$



$C(x) = -2x^2 + 80x + 100$ en un intervalo de $0 \leq x \leq 50$



7. Evaluar las funciones para los valores: $x = [0, 5, 10, 15, 20, 25, 30, 40, 50]$

$A(x) = 40x + 200$

$B(x) = 70x + 50$

$C(x) = -2x^2 + 80x + 100$

x	$A(x)$
0	200
5	400
10	600
15	800
20	1000
25	1200
30	1400
40	1800
50	2200

x	$B(x)$
0	50
5	400
10	750
15	1100
20	1450
25	1800
30	2150
40	2850
50	3550

x	$C(x)$
0	100
5	450
10	700
15	850
20	900
25	850
30	700
40	100
50	-900

8. Crear una función que determine el plan más económico para un valor de x .

vfdsd

PARTE C - Análisis

Consigna 3 — Análisis de rendimiento en un sistema distribuido

Parte A — Comprensión del modelo

1. Interpretar y explicar con sus palabras:

- Qué representa cada fila de M

Cada fila de la matriz (M) representa una función o proceso del sistema distribuido. Una fila completa indica cuánto tarda esa misma función en cada servidor, lo que sirve para ver en cuál corre mejor.

- Qué representa cada columna de M

Cada columna representa un servidor distinto. Si se observan de arriba a abajo, muestran cuánto tardan las diferentes funciones en ejecutarse en un mismo servidor, determinando si funciona correctamente o si se está quedando lenta.

- Qué representa un valor cualquiera de la matriz

Cada valor de la matriz (M) representa el tiempo promedio de ejecución medido en milisegundos, de una función en un servidor específico. Por ejemplo, el número 120 que está en la primera fila y primera columna, significa que la Función 1 tarda un promedio de 120 milisegundos en ejecutarse dentro del Servidor 1.

2. Analizar las dimensiones de M y C y determinar si es posible calcular el producto $M \cdot C$. Justificar matemáticamente.

Ambas matrices tienen 3 filas y 3 columnas. La dimensión de la primera (M) es de (3×3) y la de la segunda (C) es de (3×3) . Por lo tanto, es posible calcular $(M \cdot C)$ ya que para multiplicar matrices, la cantidad de columnas de la primera debe ser igual a la cantidad de filas de la segunda. El resultado será una nueva matriz de dimensión (3×3) .

Parte B — Implementación en Python

3. Calcular:

- El tiempo promedio de ejecución por función
- El tiempo promedio de ejecución por servidor

Primero se creó la matriz (M), donde cada fila representa una función del sistema y cada columna representa un servidor. Los valores guardados en la matriz indican los tiempos promedio de ejecución en milisegundos.

Luego, para calcular el promedio por función, se utilizó un ciclo *for* que recorre cada fila de la matriz. Dentro de ese recorrido, otro ciclo suma todos los valores de la fila y después divide el resultado entre 3 para obtener el promedio de ejecución de cada función. Finalmente, el programa muestra el promedio obtenido en pantalla.

Después, se calculó el promedio por servidor. En este caso, el programa recorre las columnas de la matriz. Para cada columna, se suman los tiempos de todas las funciones en ese servidor y luego se divide entre 3 para obtener el promedio correspondiente.

Por último, se calculó la matriz transpuesta. Para hacerlo, primero se creó una nueva matriz vacía llamada MT . Luego, usando dos ciclos *for*, se intercambiaron las filas por columnas, asignando cada valor de la matriz original en la posición inversa dentro de la nueva matriz. Finalmente, se imprimió la matriz transpuesta en pantalla.

LINK DEL REPOSITORIO:

https://github.com/caarocarranza/TP_INTEGRADOR_I/blob/main/promedio_ejecuci%C3%B3n.py

4. Calcular la matriz transpuesta de M y explicar qué representa en este contexto.

En la matriz original (M) las filas representan funciones y las columnas servidores. En la matriz transpuesta (MT) sucede al revés, las filas pasan a representar servidores mientras que las columnas pasan a representar funciones. Esto sirve para analizar de otra manera el rendimiento de cada servidor respecto a todas las funciones del sistema.

$$M^t = \begin{pmatrix} 120 & 200 & 90 \\ 150 & 180 & 110 \\ 100 & 220 & 95 \end{pmatrix}$$

Parte C — Análisis mediante producto matricial

5. Calcular el producto $T=M*C$

$$\begin{pmatrix} 120 & 150 & 100 \\ 200 & 180 & 220 \\ 90 & 110 & 95 \end{pmatrix} * \begin{pmatrix} 30 & 20 & 10 \\ 15 & 25 & 20 \\ 40 & 10 & 30 \end{pmatrix} = \begin{pmatrix} 120 * 30 + 150 * 15 + 100 * 40 & 120 * 20 + 150 * 25 + 100 * 10 & 120 * 10 + 150 * 20 + 100 * 30 \\ 200 * 30 + 180 * 15 + 220 * 40 & 200 * 20 + 180 * 25 + 220 * 10 & 200 * 10 + 180 * 20 + 220 * 30 \\ 90 * 30 + 110 * 15 + 95 * 40 & 90 * 20 + 110 * 25 + 95 * 10 & 90 * 10 + 110 * 20 + 95 * 30 \end{pmatrix} =$$

$$T = \begin{pmatrix} 9850 & 7150 & 7200 \\ 17500 & 10700 & 12200 \\ 8150 & 5500 & 5950 \end{pmatrix}$$

Parte D — Interpretación crítica

6. Analizar el resultado obtenido:

- ¿Qué podría representar cada valor de la matriz T?

Cada valor de la matriz (T) representa una combinación entre el tiempo que tarda una función y la cantidad de veces que se ejecuta. En teoría, el resultado podría interpretarse como una aproximación del tiempo total de procesamiento del sistema.

- ¿Tiene sentido físico o práctico esta operación en el contexto planteado?

La regla para multiplicar matrices nos obliga a cruzar las filas de la primera matriz con las columnas de la segunda. En nuestro caso, eso hace que terminemos multiplicando el tiempo que tarda una función en el Servidor 2 por las ejecuciones que tuvo otra función diferente en el Servidor 1, y luego sumamos todo. Como estamos mezclando datos de servidores distintos en la misma operación, el número final no representa ninguna métrica real ni útil para medir el rendimiento.

7. En caso de considerar que el producto no representa correctamente una magnitud útil:

- Explicar por qué

Matemáticamente la operación es válida, pero los valores obtenidos no representan directamente un tiempo real, un promedio ni una cantidad exacta de ejecuciones dentro del sistema. Por eso, los resultados pueden ser difíciles de interpretar desde un punto de vista práctico o físico.

- Proponer una alternativa más adecuada para combinar la información de tiempo y cantidad de ejecuciones (por ejemplo: otra operación o enfoque)

Una alternativa más adecuada sería multiplicar cada tiempo promedio de ejecución por la cantidad de ejecuciones correspondiente en el mismo servidor y función. Por ejemplo, el Servidor 1 debería multiplicarse exclusivamente por las ejecuciones de esa misma función en el Servidor 1. De esta manera, se obtendría el tiempo total consumido por cada proceso dentro del sistema, ya que permitiría identificar qué funciones consumen más recursos, cuáles generan mayor carga en los servidores y qué partes del sistema necesitan ser optimizadas.

Otra alternativa puede ser en Python, programando un bucle que multiplique de forma directa cada celda de la matriz de tiempos por la celda que ocupa el mismo lugar en la matriz de ejecuciones. De

esta manera, la matriz resultante sí nos daría como resultado los tiempos totales reales de procesamiento.

Parte E — Propiedades de la matriz

8. Determinar si la matriz M es simétrica. Justificar.

Para que una matriz sea simétrica, se tiene que cumplir que sea igual a su transpuesta. Es decir, los números se tienen que reflejar como en un espejo respecto a la diagonal principal. Al comparar los elementos posición por posición, vemos que no son iguales. Por lo tanto, la matriz (M) no es simétrica.

$$M = \begin{pmatrix} 120 & 150 & 100 \\ 200 & 180 & 220 \\ 90 & 110 & 95 \end{pmatrix} \quad M^t = \begin{pmatrix} 120 & 200 & 90 \\ 150 & 180 & 110 \\ 100 & 220 & 95 \end{pmatrix}$$

9. Evaluar si la matriz es invertible:

- Calcular su determinante

Para encontrar el determinante de la matriz se aplica el método de Sarrus, que consiste en tomar los valores de la primera y segunda fila y agregarlos por debajo de la matriz 3x3. Esta serie de pasos da como resultado una matriz de 5x3, en la cual se calculan los productos de las diagonales principales y de las diagonales secundarias, restando el total de las secundarias al total de las principales.

Diagonales principales:

$$(120 \times 180 \times 95) = 2.052.000$$

$$(150 \times 220 \times 90) = 2.970.000$$

$$(100 \times 200 \times 110) = 2.200.000$$

$$\text{Suma: } 2.052.000 + 2.970.000 + 2.200.000 = 7.222.000$$

Diagonales secundarias:

$$(100 \times 180 \times 90) = 1.620.000$$

$$(120 \times 220 \times 110) = 2.904.000$$

$$(150 \times 200 \times 95) = 2.850.000$$

$$\text{Suma: } 1.620.000 + 2.904.000 + 2.850.000 = 7.374.000$$

$$\text{Determinante (M)} = 7.222.000 - 7.374.000 = -152.000$$

- Indicar si es posible obtener la inversa

Sí, es posible obtener la inversa. Una matriz es invertible si y sólo si su determinante es distinto de cero, como nuestro determinante dio -152.000, confirmamos que la matriz (M) sí tiene inversa.

10. Interpretar:

- ¿Qué implicaría que la matriz no sea invertible en términos de la información que representa?

Si la matriz no fuese invertible, significaría que parte de la información es dependiente o redundante, es decir, que algunos datos se repiten o dependen de otros.

En este contexto, podría indicar que algunos servidores tienen comportamientos muy parecidos y no aportan nueva información. Además, si no fuese posible calcular la matriz inversa, no se podrían realizar ciertos cálculos y análisis sobre el rendimiento del sistema.

Parte F — Análisis aplicado

11. Determinar:

- Qué función presenta mayor costo computacional promedio

La función que presenta el mayor costo computacional promedio es la Función 2, ya que posee un tiempo promedio de ejecución de 200 milisegundos. Esto indica que es la que más tarda en procesarse y la que más recursos consume dentro del sistema.

Promedio por función:

Función 1 = 123.33 milisegundos

Función 2 = 200.00 milisegundos

Función 3 = 98.33 milisegundos

- Qué servidor resulta más eficiente

Por otro lado, el servidor más eficiente es el Servidor 1 porque tiene un menor tiempo de ejecución de 136,67 milisegundos. Por lo tanto, ejecutará las funciones más rápido que los demás servidores ofreciendo un mejor rendimiento general.

Promedio por servidor:

Servidor 1 = 136.67 milisegundos

Servidor 2 = 146.67 milisegundos

Servidor 3 = 138.34 milisegundos

12. Proponer una recomendación técnica basada en los resultados obtenidos (por ejemplo: redistribución de carga, optimización de funciones, etc.):

Basándonos en los promedios calculados, el equipo propone dos recomendaciones:

La primera consiste en revisar el código de la Función 2. Debido a que es la que más tiempo tarda (200.00 ms), el objetivo sería analizar sus bucles o condiciones en Python para ver si se puede simplificar el código y lograr que se ejecute más rápido.

La segunda recomendación consiste en reasignar las tareas pesadas al servidor más rápido. Los resultados indican que el Servidor 1 es el más rápido de todos (136.67 ms) y el Servidor 2 es el más lento (146.67 ms). Por lo tanto, sugerimos redistribuir las tareas: pasar la mayor parte de las

ejecuciones de la Función 2 (que es la más pesada) al Servidor 1. De esta manera, se aprovecharía la computadora que mejor rinde para procesar el trabajo más difícil, aliviando al servidor más lento.

Link repositorio:

Link video: